



南開大學
Nankai University

数据结构实验报告二

链表

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网路空间安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	2
1.4 代码	2
1.5 测试用例截图	8
1.6 复杂度分析	8
1.6.1 空间复杂度为 $O(n + m)$	8
1.6.2 时间复杂度为 $O(n + m)$	8
2 题目二	9
2.1 题目表述	9
2.2 代码 2 的测试用例	9
2.3 思路	9
2.4 代码	10
2.5 测试用例截图	17
2.6 复杂度分析	17
2.6.1 空间复杂度	17
2.6.2 时间复杂度 $O(n)$	17

1 题目一

1.1 题目表述

令 a 和 b 的类型为 `extendedChain`。

1. 编写一个非成员方法 `meld`，它生成一个新的扩展的链表 c ，它从 a 的首元素开始，交替地包含 a 和 b 的元素。如果一个链表的元素取完了，就把另一个链表的剩余元素附加到新的扩展链表 c 中。方法的复杂度应与链表 a 和 b 的长度具有线性关系。
2. 证明方法具有线性复杂度。
3. 使用自己的测试数据检验方法的正确性。

1.2 代码 1 的测试用例

1. $a = []$, $b = [] \rightarrow c = []$
2. $a = []$, $b = [1,2,3] \rightarrow c = [1,2,3]$
3. $a = [7,8]$, $b = [] \rightarrow c = [7,8]$
4. $a = [1,3,5]$, $b = [2,4,6] \rightarrow c = [1,2,3,4,5,6]$
5. $a = [10,20,30,40]$, $b = [11] \rightarrow c = [10,11,20,30,40]$
6. $a = [-3,0,5]$, $b = [-2,-1,6,7] \rightarrow c = [-3,-2,0,-1,5,6,7]$

1.3 思路

`meld` 方法的实现采用了交替合并的思路，从链表 a 的首元素开始，交替地从 a 和 b 中取元素添加到新链表 c 中，当某个链表先耗尽时，将另一个链表的剩余元素全部追加到 c 的末尾。

节点处理方式新建节点：创建新链表 c 作为结果容器，使用双指针同步遍历链表 a 和 b ，交替将 a 和 b 的元素添加到 c 中，最后处理完剩余数据后返回合并后的新链表 c 。

1.4 代码

```
1 #include <iostream>
2 #include <vector>
3 #include <iomanip>
4 #include<sstream>
5 using namespace std;
6 template<typename T>
7 struct extendedChainNode {
8     T element;
9     extendedChainNode<T>* next;
10     extendedChainNode() : next(nullptr) {}
11     extendedChainNode(const T& element) : element(element), next(nullptr) {}
```

```
12     extendedChainNode(const T& element, extendedChainNode<T>* next)
13         : element(element), next(next) {}
14 };
15
16 template<typename T>
17 class extendedChain {
18 private:
19     extendedChainNode<T>* firstNode;
20     extendedChainNode<T>* lastNode;
21     int listSize;
22 public:
23     extendedChain() : firstNode(nullptr), lastNode(nullptr), listSize(0) {}
24     ~extendedChain() {
25         clear();
26     }
27
28     extendedChain(const extendedChain<T>& other) {
29         firstNode = lastNode = nullptr;
30         listSize = 0;
31         extendedChainNode<T>* current = other.firstNode;
32         while (current != nullptr) {
33             push_back(current->element);
34             current = current->next;
35         }
36     }
37
38     extendedChain<T>& operator=(const extendedChain<T>& other) {
39         if (this != &other) {
40             clear();
41             extendedChainNode<T>* current = other.firstNode;
42             while (current != nullptr) {
43                 push_back(current->element);
44                 current = current->next;
45             }
46         }
47         return *this;
48     }
49
50     void clear() {
51         while (firstNode != nullptr) {
52             extendedChainNode<T>* nextNode = firstNode->next;
53             delete firstNode;
```

```
54     firstNode = nextNode;
55 }
56 lastNode = nullptr;
57 listSize = 0;
58 }
59
60 void push_back(const T& element) {
61     extendedChainNode<T>* newNode = new extendedChainNode<T>(element);
62     if (lastNode == nullptr) {
63         firstNode = lastNode = newNode;
64     }
65     else {
66         lastNode->next = newNode;
67         lastNode = newNode;
68     }
69     listSize++;
70 }
71
72 int size() const {
73     return listSize;
74 }
75
76 bool empty() const {
77     return listSize == 0;
78 }
79
80 extendedChainNode<T>* getFirst() const {
81     return firstNode;
82 }
83
84 void print() const {
85     cout << "[";
86     extendedChainNode<T>* current = firstNode;
87     while (current != nullptr) {
88         cout << current->element;
89         if (current->next != nullptr) {
90             cout << ", ";
91         }
92         current = current->next;
93     }
94     cout << "]";
95 }
```

```
96
97 void fromVector(const vector<T>& vec) {
98     clear();
99     for (const T& element : vec) {
100         push_back(element);
101     }
102 }
103
104 vector<T> toVector() const {
105     vector<T> result;
106     extendedChainNode<T>* current = firstNode;
107     while (current != nullptr) {
108         result.push_back(current->element);
109         current = current->next;
110     }
111     return result;
112 }
113
114 friend extendedChain<T> meld(const extendedChain<T>& a, const extendedChain<T>&
115     b) {
116     extendedChain<T> c;
117     extendedChainNode<T>* currentA = a.firstNode;
118     extendedChainNode<T>* currentB = b.firstNode;
119
120     while (currentA != nullptr && currentB != nullptr) {
121         c.push_back(currentA->element);
122         c.push_back(currentB->element);
123         currentA = currentA->next;
124         currentB = currentB->next;
125     }
126
127     while (currentA != nullptr) {
128         c.push_back(currentA->element);
129         currentA = currentA->next;
130     }
131
132     while (currentB != nullptr) {
133         c.push_back(currentB->element);
134         currentB = currentB->next;
135     }
136 }
```

```
137
138     return c;
139 }
140 };
141
142
143 void runTests() {
144     cout << "测试结果表格: " << endl;
145     cout << string(120, '-') << endl;
146     cout << setw(25) << "输入 a" << setw(25) << "输入 b"
147         << setw(25) << "程序输出 c" << setw(25) << "期望输出"
148         << setw(15) << "是否正确" << endl;
149     cout << string(120, '-') << endl;
150
151     // 测试用例集合
152     vector<pair<vector<int>, vector<int>>> testCases = {
153         {}, {},
154         {}, {1, 2, 3},
155         {{7, 8}, {}},
156         {{1, 3, 5}, {2, 4, 6}},
157         {{10, 20, 30, 40}, {11}},
158         {{-3, 0, 5}, {-2, -1, 6, 7}}
159     };
160
161     // 期望输出
162     vector<vector<int>> expectedOutputs = {
163         {},
164         {1, 2, 3},
165         {7, 8},
166         {1, 2, 3, 4, 5, 6},
167         {10, 11, 20, 30, 40},
168         {-3, -2, 0, -1, 5, 6, 7}
169     };
170
171     for (size_t i = 0; i < testCases.size(); ++i) {
172         extendedChain<int> a, b;
173         a.fromVector(testCases[i].first);
174         b.fromVector(testCases[i].second);
175
176         extendedChain<int> c = meld(a, b);
177         vector<int> result = c.toVector();
178         vector<int> expected = expectedOutputs[i];
```

```
179
180     auto vectorToString = [](const vector<int>& vec) -> string {
181         string result = "[";
182         for (size_t j = 0; j < vec.size(); ++j) {
183             result += to_string(vec[j]);
184             if (j < vec.size() - 1) {
185                 result += ", ";
186             }
187         }
188         result += "]";
189         return result;
190     };
191     string aStr = vectorToString(testCases[i].first);
192     string bStr = vectorToString(testCases[i].second);
193     string resultStr = vectorToString(result);
194     string expectedStr = vectorToString(expected);
195     string correct = (result == expected) ? "是" : "否";
196     cout << setw(25) << aStr << setw(25) << bStr
197         << setw(25) << resultStr << setw(25) << expectedStr
198         << setw(15) << correct << endl;
199 }
200 cout << string(120, '-') << endl;
201 }
202
203 int main() {
204     runTests();
205     system("pause");
206     return 0;
207 }
```


1.5 测试用例截图

测试结果表格：

输入 a	输入 b	程序输出 c	期望输出	是否正确
[]	[]	[]	[]	是
[7, 8]	[1, 2, 3]	[1, 2, 3]	[1, 2, 3]	是
[1, 3, 5]	[2, 4, 6]	[1, 2, 3, 4, 5, 6]	[1, 2, 3, 4, 5, 6]	是
[10, 20, 30, 40]	[11]	[10, 11, 20, 30, 40]	[10, 11, 20, 30, 40]	是
[-3, 0, 5]	[-2, -1, 6, 7]	[-3, -2, 0, -1, 5, 6, 7]	[-3, -2, 0, -1, 5, 6, 7]	是

请按任意键继续 . . . |

图 1.1: 题目一测试用例输出结果

1.6 复杂度分析

证明方法具有线性复杂度 (说明为什么是 $O(n+m)$):

1.6.1 空间复杂度为 $O(n + m)$

新建了 $n + m$ 个节点来存储合并后的链表。

1.6.2 时间复杂度为 $O(n + m)$

主循环部分

```

1 while (currentA != nullptr && currentB != nullptr) {
2     c.push_back(currentA->element); // O(1)
3     c.push_back(currentB->element); // O(1)
4     currentA = currentA->next;    // O(1)
5     currentB = currentB->next;    // O(1)
6 }
```

$$4 \times \min(n, m) = O(\min(n, m))$$

其他部分:

```

1 while (currentA != nullptr) { // 最多执行 |n - m| 次
2     c.push_back(currentA->element); // O(1)
3     currentA = currentA->next;    // O(1)
```

```
4 }
5
6 while (currentB != nullptr) { // 最多执行 |n - m| 次
7     c.push_back(currentB->element); // O(1)
8     currentB = currentB->next;    // O(1)
9 }
```

$$2 \times |n - m| = O(|n - m|)$$

由于 $\min(n, m) + |n - m| = \max(n, m)$ $n + m$, 所以总体时间复杂度: $O(\min(n, m)) + O(|n - m|) = O(n + m)$

2 题目二

2.1 题目表述

使用带有头节点的双向循环链表完成:

1. 令 c 的类型为扩展链表 `extendedChain`。

(a) 编写一个非成员方法 `split(a,b)`, 它生成两个扩展链表 a 和 b 。 a 包含 c 中索引为奇数的元素, b 包含 c 中其余的元素。这个方法不能改变 c 。

(b) 计算方法的复杂度。

(c) 使用自己的测试数据检验方法的正确性。

2. 编写方法 `chain<T>::split`, 它与上题的函数类似。然而, 它用输入链表 `*this` 的空间建立了链表 a 和 b 。

2.2 代码 2 的测试用例

1. $c = [] \rightarrow a = [], b = []$

2. $c = [7] \rightarrow a = [7], b = []$

3. $c = [1,2] \rightarrow a = [1], b = [2]$

4. $c = [1,2,3,4,5] \rightarrow a = [1,3,5], b = [2,4]$

5. $c = [10,20,30,40] \rightarrow a = [10,30], b = [20,40]$

6. $c = [-1,-1,0,7] \rightarrow a = [-1,0], b = [-1,7]$

2.3 思路

1. 对非成员 `split`: **原链表保持不变**, 依据索引奇偶性, 把元素添加到新创建的链表 a 和 b 中。

2. 对成员 `split`: 清空原链表, 通过”**摘链**”操作将原链表节点直接移动到 a 和 b , 复用原链表的节点空间。

2.4 代码

```
1 #include <iostream>
2 #include <vector>
3 #include <iomanip>
4 #include <sstream>
5 using namespace std;
6 template<typename T>
7 struct chainNode {
8     T element;
9     chainNode<T>* next;
10    chainNode<T>* prev;
11
12    chainNode(const T& element = T(), chainNode<T>* next = nullptr, chainNode<T>*
        prev = nullptr)
13        : element(element), next(next), prev(prev) {}
14 };
15
16 template<typename T>
17 class extendedChain {
18 private:
19     chainNode<T>* head;
20     int listSize;
21 public:
22     extendedChain() {
23         head = new chainNode<T>();
24         head->next = head;
25         head->prev = head;
26         listSize = 0;
27     }
28     extendedChain(const extendedChain<T>& other) {
29         head = new chainNode<T>();
30         head->next = head;
31         head->prev = head;
32         listSize = 0;
33
34         chainNode<T>* current = other.head->next;
35         while (current != other.head) {
36             push_back(current->element);
37             current = current->next;
38         }
39     }
```

```
40     extendedChain<T>& operator=(const extendedChain<T>& other) {
41         if (this != &other) {
42             clear();
43             chainNode<T>* current = other.head->next;
44             while (current != other.head) {
45                 push_back(current->element);
46                 current = current->next;
47             }
48         }
49         return *this;
50     }
51     ~extendedChain() {
52         clear();
53         delete head;
54     }
55     void clear() {
56         chainNode<T>* current = head->next;
57         while (current != head) {
58             chainNode<T>* nextNode = current->next;
59             delete current;
60             current = nextNode;
61         }
62         head->next = head;
63         head->prev = head;
64         listSize = 0;
65     }
66     void push_back(const T& element) {
67         chainNode<T>* newNode = new chainNode<T>(element, head, head->prev);
68         head->prev->next = newNode;
69         head->prev = newNode;
70         listSize++;
71     }
72     int size() const {
73         return listSize;
74     }
75     bool empty() const {
76         return listSize == 0;
77     }
78     T get(int index) const {
79         if (index < 0 || index >= listSize) {
80             throw out_of_range("Index out of range");
81         }
82     }
```

```
82     chainNode<T>* current = head->next;
83     for (int i = 0; i < index; i++) {
84         current = current->next;
85     }
86     return current->element;
87 }
88 void set(int index, const T& element) {
89     if (index < 0 || index >= listSize) {
90         throw out_of_range("Index out of range");
91     }
92     chainNode<T>* current = head->next;
93     for (int i = 0; i < index; i++) {
94         current = current->next;
95     }
96     current->element = element;
97 }
98 void print() const {
99     cout << "[";
100    chainNode<T>* current = head->next;
101    while (current != head) {
102        cout << current->element;
103        if (current->next != head) {
104            cout << ",";
105        }
106        current = current->next;
107    }
108    cout << "]";
109 }
110 void fromVector(const vector<T>& vec) {
111     clear();
112     for (const T& element : vec) {
113         push_back(element);
114     }
115 }
116 vector<T> toVector() const {
117     vector<T> result;
118     chainNode<T>* current = head->next;
119     while (current != head) {
120         result.push_back(current->element);
121         current = current->next;
122     }
123     return result;
```

```
124     }
125
126     friend void split(const extendedChain<T>& c, extendedChain<T>& a,
127         extendedChain<T>& b) {
128         a.clear();
129         b.clear();
130
131         chainNode<T>* current = c.head->next;
132         int index = 0;
133         while (current != c.head) {
134             if (index % 2 == 0) {
135                 a.push_back(current->element);
136             }
137             else {
138                 b.push_back(current->element);
139             }
140             current = current->next;
141             index++;
142         }
143
144     void split(extendedChain<T>& a, extendedChain<T>& b) {
145         a.clear();
146         b.clear();
147         if (empty()) return;
148         chainNode<T>* current = head->next;
149         int index = 0;
150         while (current != head) {
151             chainNode<T>* nextNode = current->next;
152             current->prev->next = current->next;
153             current->next->prev = current->prev;
154             if (index % 2 == 0) {
155                 current->prev = a.head->prev;
156                 current->next = a.head;
157                 a.head->prev->next = current;
158                 a.head->prev = current;
159                 a.listSize++;
160             }
161             else {
162                 current->prev = b.head->prev;
163                 current->next = b.head;
164                 b.head->prev->next = current;
```

```
165         b.head->prev = current;
166         b.listSize++;
167     }
168     current = nextNode;
169     index++;
170 }
171 head->next = head;
172 head->prev = head;
173 listSize = 0;
174 }
175 };
176
177 string vecToString(const vector<int>& v) {
178     ostreamstream oss;
179     oss << "[";
180     for (int j = 0; j < v.size(); ++j) {
181         oss << v[j];
182         if (j < v.size() - 1) oss << ",";
183     }
184     oss << "]";
185     return oss.str();
186 }
187
188 void testSplitFunctions() {
189     cout << "测试非成员方法 split (不改变原链表) : " << endl;
190     cout << string(95, '-') << endl;
191     cout << left
192         << setw(15) << "输入 c"
193         << setw(20) << "程序输出 a"
194         << setw(20) << "程序输出 b"
195         << setw(20) << "期望输出 a"
196         << setw(20) << "期望输出 b"
197         << "是否正确" << endl;
198
199     cout << string(95, '-') << endl;
200
201     // 测试用例
202     vector<vector<int>> testCases = {
203         {}, // c = []
204         {7}, // c = [7]
205         {1, 2}, // c = [1,2]
206         {1, 2, 3, 4, 5}, // c = [1,2,3,4,5]
```

```
207     {10, 20, 30, 40}, // c = [10,20,30,40]
208     {-1, -1, 0, 7} // c = [-1,-1,0,7]
209 };
210
211 vector<vector<int>> expectedA = {
212     {}, {7}, {1}, {1, 3, 5}, {10, 30}, {-1, 0}
213 };
214
215 vector<vector<int>> expectedB = {
216     {}, {}, {2}, {2, 4}, {20, 40}, {-1, 7}
217 };
218
219 for (int i = 0; i < testCases.size(); i++) {
220     extendedChain<int> c, a, b;
221     c.fromVector(testCases[i]);
222     split(c, a, b);
223
224     vector<int> actualA = a.toVector();
225     vector<int> actualB = b.toVector();
226
227     bool correct = (actualA == expectedA[i] && actualB == expectedB[i]);
228
229     cout << left
230         << setw(15) << vecToString(testCases[i])
231         << setw(20) << vecToString(actualA)
232         << setw(20) << vecToString(actualB)
233         << setw(20) << vecToString(expectedA[i])
234         << setw(20) << vecToString(expectedB[i])
235         << (correct ? "是" : "否") << endl;
236 }
237 cout << string(95, '-') << endl;
238 cout << " 测试成员方法 split (使用原链表空间, 摘链) : " << endl;
239 cout << string(95, '-') << endl;
240 cout << left
241     << setw(15) << "原链表 c"
242     << setw(20) << "输出 a"
243     << setw(20) << "输出 b"
244     << setw(20) << "期望 a"
245     << setw(20) << "期望 b"
246     << "c为空?" << endl;
247 cout << string(95, '-') << endl;
248 for (int i = 0; i < testCases.size(); i++) {
```



```
249     extendedChain<int> c, a, b;
250     c.fromVector(testCases[i]);
251     c.split(a, b);
252     vector<int> actualA = a.toVector();
253     vector<int> actualB = b.toVector();
254     vector<int> actualC = c.toVector();
255     bool correct = (actualA == expectedA[i] && actualB == expectedB[i] &&
        actualC.empty());
256     string cEmptyStr = actualC.empty() ? "是" : "否";
257     cout << left
258         << setw(15) << vecToString(testCases[i])
259         << setw(20) << vecToString(actualA)
260         << setw(20) << vecToString(actualB)
261         << setw(20) << vecToString(expectedA[i])
262         << setw(20) << vecToString(expectedB[i])
263         << cEmptyStr << endl;
264 }
265 cout << string(95, '-') << endl;
266 }
267
268 int main() {
269     testSplitFunctions();
270     system("pause");
271     return 0;
272 }
```

2.5 测试用例截图

```
C:\Users\ASUS\source\repos\c × + v
测试非成员方法 split (不改变原链表) :
-----
输入 c      程序输出 a      程序输出 b      期望输出 a      期望输出 b      是否正确
-----
[]          []          []          []          []          是
[7]         [7]         [7]         [7]         [7]         是
[1,2]       [1]         [2]         [1]         [2]         是
[1,2,3,4,5] [1,3,5]     [2,4]       [1,3,5]     [2,4]       是
[10,20,30,40] [10,30]    [20,40]     [10,30]     [20,40]     是
[-1,-1,0,7] [-1,0]      [-1,7]      [-1,0]      [-1,7]      是
-----
测试成员方法 split (使用原链表空间, 摘链) :
-----
原链表 c    输出 a      输出 b      期望 a      期望 b      c为空?
-----
[]          []          []          []          []          是
[7]         [7]         [7]         [7]         [7]         是
[1,2]       [1]         [2]         [1]         [2]         是
[1,2,3,4,5] [1,3,5]     [2,4]       [1,3,5]     [2,4]       是
[10,20,30,40] [10,30]    [20,40]     [10,30]     [20,40]     是
[-1,-1,0,7] [-1,0]      [-1,7]      [-1,0]      [-1,7]     是
-----
请按任意键继续 . . . |
```

图 2.2: 题目二测试用例输出结果

2.6 复杂度分析

2.6.1 空间复杂度

1. 对非成员 split: 通过遍历原链表 c, 将节点按索引奇偶性分别插入新链表 a 和 b 中, 对每个元素调用 push_back, 动态创建新的节点, 需要额外分配 $O(n)$ 个新节点来存储数据。空间复杂度为 $O(n)$ 。
2. 对成员 split: 直接操作原链表 *this 的节点, 通过修改节点的前驱 (prev) 和后继 (next) 指针, 将原节点“摘下”并重新链接到链表 a 和 b 中, 无需新建节点, 空间复杂度为 $O(1)$ 。

2.6.2 时间复杂度 $O(n)$

非成员函数的 split 和成员函数的 split, 都需要遍历原链表一次。令原链表长度为 n, 每个节点的处理为 $O(1)$ 操作。可知时间复杂度均为 $O(n)$ 。